



DSA0602-DATA HANDLING AND DATA VISUALIZATION

SLOT A

Dr. ALMAS BEGUM

PRESENTED BY:

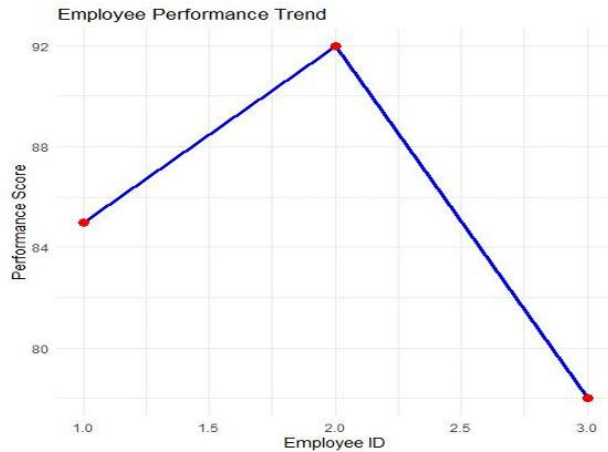
SURU SAI KEERTHI [192424368]

Set 8 Employee Performance Analysis

8.A: Create a line chart to visualize the performance trend of employees over time. Label the axes and title the chart.

```
employee <- data.frame(  
  EmployeeID = c(1, 2, 3, 4, 5),  
  Time = c(1, 2, 3, 4, 5),  
  PerformanceScore = c(75, 82, 78, 88, 92)  
)  
  
plot(  
  employee$Time,  
  employee$PerformanceScore,  
  type = "o",  
  pch = 16,  
  xlab = "Time",  
  ylab = "Performance Score",  
  main = "Employee Performance Trend Over Time"  
)  
  
text(  
  employee$Time,  
  employee$PerformanceScore,  
  labels = paste("Employee", employee$EmployeeID),  
  pos = 3
```

)



8.B: Generate a bar chart showing the distribution of employees across different departments. Label the chart elements.

```
employee <- data.frame(  
  EmployeeID = c(1, 2, 3, 4, 5),  
  Department = c("Sales", "HR", "Marketing", "Sales", "HR")  
)
```

)

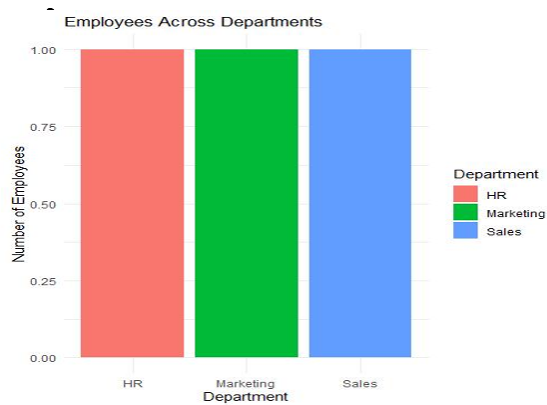
```
department_count <- table(employee$Department)
```

```
bars <- barplot(  
  department_count,  
  xlab = "Department",  
  ylab = "Number of Employees",  
  main = "Distribution of Employees Across Departments"
```

)

```
text(  
  x = bars,  
  y = department_count,  
  labels = department_count,  
  pos = 3
```

)



8.C: Build a table to display the performance data for each employee. Label the table elements.

```
employee <- data.frame(
  Employee_ID = c(1, 2, 3, 4, 5),
  Department = c("Sales", "HR", "Marketing", "Sales", "HR"),
  Years_of_Service = c(5, 3, 7, 4, 2),
  Performance_Score = c(85, 92, 78, 90, 76)
)
print("Employee Performance Data")
print(employee)
```

EmployeeID Department YearsService Performance				
1	1	Sales	5	85
2	2	HR	3	92
3	3	Marketing	7	78

8.D: Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of employee performance data.

```
library(shiny)
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1, 2, 3),
  Department = c("Sales", "HR", "Marketing"),
  YearsService = c(5, 3, 7),
  PerformanceScore = c(85, 92, 78)
)
```

```
ui <- fluidPage(
```

```

titlePanel("Employee Performance Dashboard"),
sidebarLayout(
  sidebarPanel(
    selectInput(
      "department",
      "Select Department:",
      choices = c("All", unique(employee_data$Department))
    )
  ),
  mainPanel(
    plotOutput("lineChart"),
    plotOutput("barChart"),
    tableOutput("employeeTable")
  )
)
)
server <- function(input, output) {
  filtered_data <- reactive({
    if (input$department == "All") {
      employee_data
    } else {
      subset(employee_data, Department == input$department)
    }
  })
  output$lineChart <- renderPlot({
    ggplot(
      filtered_data(),
      aes(x = YearsService, y = PerformanceScore, group = 1)
    ) +
    geom_line() +
    geom_point(size = 3) +

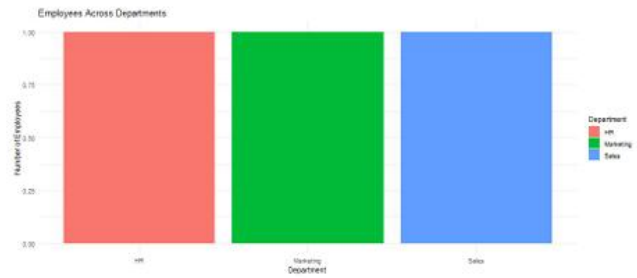
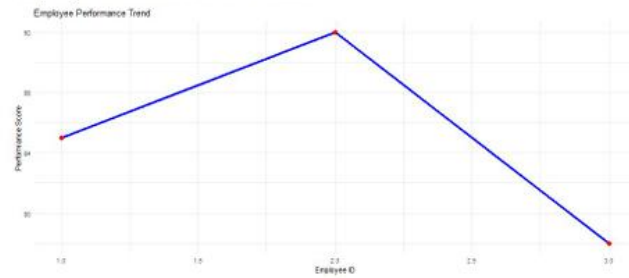
```

```

    labs(
      title = "Employee Performance Trend",
      x = "Years of Service",
      y = "Performance Score"
    ) +
    theme_minimal()
  })
output$barChart <- renderPlot({
  ggplot(
    filtered_data(),
    aes(x = Department)
  ) +
    geom_bar() +
    labs(
      title = "Employees Across Departments",
      x = "Department",
      y = "Number of Employees"
    ) +
    theme_minimal()
  })
output$employeeTable <- renderTable({
  filtered_data()
})
}
shinyApp(ui = ui, server = server)

```

Employee Performance Dashboard



Employee Performance Table

EmployeeID	Department	YearsService	Performance
1.00	Sales	5.00	85.00
2.00	HR	3.00	92.00
3.00	Marketing	7.00	78.00

set 9

9.A: Import the dataset in R. Plot a histogram for Quiz_Score. Draw a boxplot of Quiz_Score by Course. Add title and labels. Interpret student performance.

```
student_data <- data.frame(
  Student_ID = c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10),
  Course = c("Python", "R", "Python", "R", "Python",
             "R", "Python", "R", "Python", "R"),
  Quiz_Score = c(85, 78, 92, 74, 88, 81, 95, 76, 89, 84)
)

print(student_data)

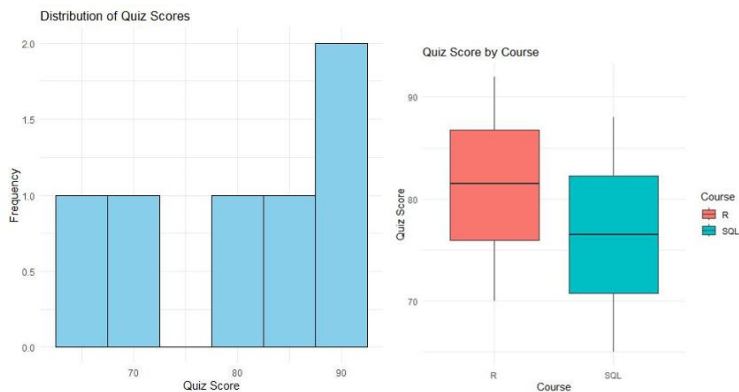
hist(
  student_data$Quiz_Score,
  main = "Distribution of Quiz Scores",
  xlab = "Quiz Score",
  ylab = "Frequency"
)

boxplot(
  Quiz_Score ~ Course,
  data = student_data,
  main = "Quiz Scores by Course",
  xlab = "Course",
  ylab = "Quiz Score"
)
```

```

print("Interpretation:")
print("The histogram shows the overall distribution of student quiz scores.")
print("Most students achieved moderate to high scores.")
print("The boxplot compares student performance between the Python and R courses.")
print("It helps identify differences in median scores, score variability, and possible outliers between courses.")

```



9.B: Create a scatter plot using R programming between Study_Time and Quiz_Score. Represent Videos_Watched using bubble size. Apply transparency. Explain the learning pattern.

```

student_data <- data.frame(
  Student_ID = c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10),
  Study_Time = c(2, 3, 5, 4, 6, 3, 7, 5, 8, 6),
  Quiz_Score = c(65, 70, 82, 75, 88, 72, 92, 84, 96, 90),
  Videos_Watched = c(2, 3, 5, 4, 6, 3, 7, 5, 8, 6)
)

plot(
  student_data$Study_Time,
  student_data$Quiz_Score,
  pch = 16,
  cex = student_data$Videos_Watched / 2,
  col = rgb(0, 0, 1, 0.5),
  xlab = "Study Time",
  ylab = "Quiz Score",
  main = "Study Time vs Quiz Score"
)

```

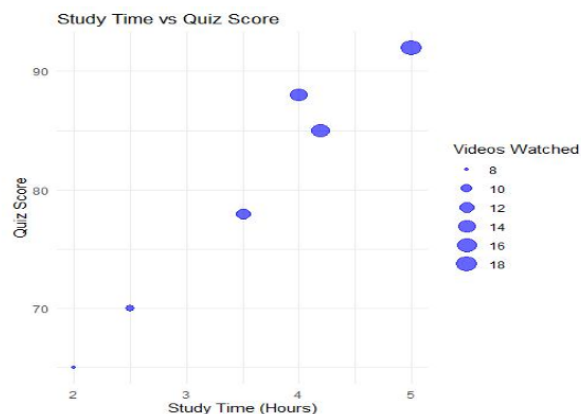
```

legend(
  "topleft",
  legend = "Bubble Size = Videos Watched",
  pch = 16,
  pt.cex = 2
)

print("Learning Pattern:")

```

print("The scatter plot shows a positive relationship between study time and quiz scores. Students who spend more time studying generally achieve higher quiz scores. Larger bubbles represent students who watched more videos, suggesting that increased engagement with learning videos may also contribute to better performance.")



9.C:Using R programming, convert Login_Date into Date format. Calculate average quiz score per month. Plot a line chart. Apply moving average smoothing. Identify the trend.

```

student_data <- data.frame(
  Login_Date = c(
    "2025-01-05", "2025-01-15",
    "2025-02-10", "2025-02-20",
    "2025-03-05", "2025-03-18",
    "2025-04-08", "2025-04-22",
    "2025-05-10", "2025-05-25"
  ),
  Quiz_Score = c(65, 70, 72, 75, 78, 82, 84, 87, 90, 94)
)

```



```

student_data$Login_Date <- as.Date(student_data$Login_Date)

student_data$Month <- format(
  student_data$Login_Date,
  "%Y-%m"
)

monthly_avg <- aggregate(
  Quiz_Score ~ Month,
  data = student_data,
  FUN = mean
)

monthly_avg$Moving_Average <- stats::filter(
  monthly_avg$Quiz_Score,
  rep(1/3, 3),
  sides = 1
)

plot(
  monthly_avg$Quiz_Score,
  type = "o",
  pch = 16,
  xaxt = "n",
  xlab = "Month",
  ylab = "Average Quiz Score",
  main = "Monthly Average Quiz Score with Moving Average"
)

axis(
  1,
  at = 1:nrow(monthly_avg),
  labels = monthly_avg$Month
)

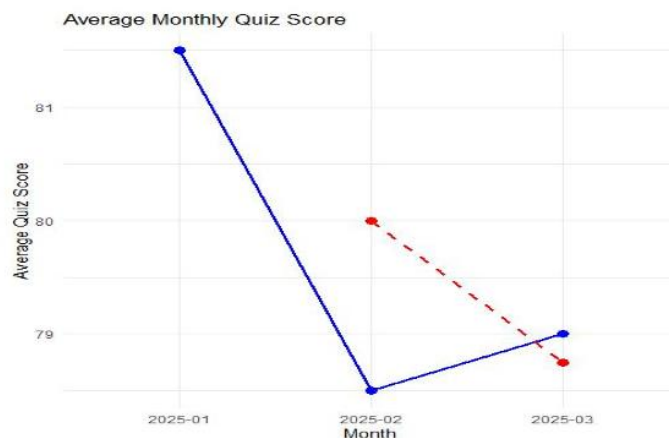
lines(
  monthly_avg$Moving_Average,

```

```

type = "o",
pch = 16,
lty = 2
)
legend(
  "topleft",
  legend = c("Average Quiz Score", "3-Month Moving Average"),
  lty = c(1, 2),
  pch = c(16, 16)
)
print(monthly_avg)
print("Trend:")
print("The monthly average quiz scores show an increasing trend over time. The moving average
smooths short-term fluctuations and clearly indicates gradual improvement in student performance.")

```



9.D: Import Online_Learning_Activity.csv into Tableau. Create a bar chart showing average quiz score by Course. Create a scatter plot (Study Time vs Quiz Score). Add filter for Gender and Course. Design a mini interactive dashboard.

```

library(shiny)

data <- read.csv("Online_Learning_Activity.csv")

ui <- fluidPage(
  titlePanel("Online Learning Activity Dashboard"),

```

```

sidebarLayout(
  sidebarPanel(
    selectInput(
      "gender",
      "Select Gender:",
      choices = c("All", unique(data$Gender))
    ),

    selectInput(
      "course",
      "Select Course:",
      choices = c("All", unique(data$Course))
    )
  ),

  mainPanel(
    plotOutput("barChart"),
    plotOutput("scatterPlot")
  )
)

server <- function(input, output) {

  filtered_data <- reactive({
    d <- data

    if (input$gender != "All") {
      d <- d[d$Gender == input$gender, ]
    }
  })
}

```

```
if (input$course != "All") {  
  d <- d[d$Course == input$course, ]  
}
```

```
d  
})
```

```
output$barChart <- renderPlot({
```

```
  avg_score <- aggregate(  
    Quiz_Score ~ Course,  
    data = filtered_data(),  
    FUN = mean  
  )
```

```
  barplot(  
    avg_score$Quiz_Score,  
    names.arg = avg_score$Course,  
    main = "Average Quiz Score by Course",  
    xlab = "Course",  
    ylab = "Average Quiz Score"  
  )  
})
```

```
output$scatterPlot <- renderPlot({
```

```
  plot(  
    filtered_data()$Study_Time,  
    filtered_data()$Quiz_Score,  
    pch = 16,
```

```

    xlab = "Study Time",
    ylab = "Quiz Score",
    main = "Study Time vs Quiz Score"
  )
})
}

shinyApp(ui = ui, server = server)

```



SET 10:

10.A: Create a grouped bar chart to visualize the distribution of answers for Question 1. Label the chart elements.

```

survey_data <- data.frame(
  Gender = c("Male", "Female", "Male", "Female", "Male",
             "Female", "Male", "Female", "Male", "Female"),
  Question1 = c("Strongly Agree", "Agree", "Agree", "Strongly Agree",
                "Neutral", "Agree", "Neutral", "Agree",
                "Strongly Agree", "Neutral")
)

answer_table <- table(
  survey_data$Question1,
  survey_data$Gender
)

barplot(

```

```

answer_table,
beside = TRUE,
legend.text = TRUE,
xlab = "Gender",
ylab = "Number of Responses",
main = "Distribution of Answers for Question 1")

```



10.B: Generate a stacked bar chart to represent the overall distribution of responses for all three questions.

```

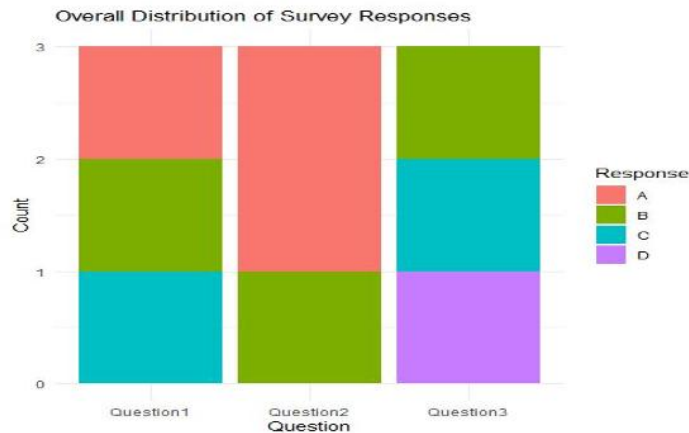
survey_data <- data.frame(
  Question1 = c("Agree", "Strongly Agree", "Neutral", "Agree", "Strongly Agree",
    "Agree", "Neutral", "Agree", "Strongly Agree", "Neutral"),
  Question2 = c("Neutral", "Agree", "Agree", "Strongly Agree", "Agree",
    "Neutral", "Agree", "Strongly Agree", "Agree", "Neutral"),
  Question3 = c("Strongly Agree", "Agree", "Neutral", "Agree", "Neutral",
    "Strongly Agree", "Agree", "Agree", "Neutral", "Agree")
)
response_levels <- c("Neutral", "Agree", "Strongly Agree")
response_counts <- sapply(
  survey_data,
  function(x) table(factor(x, levels = response_levels))
)
barplot(
  response_counts,
  beside = FALSE,
  legend.text = response_levels,

```

```

xlab = "Questions",
ylab = "Number of Responses",
main = "Overall Distribution of Responses for All Three Questions"
)

```



10.C: Build a table to show the survey response data for each survey. Label the table elements.

```

survey_data <- data.frame(
  Survey_ID = c(1, 2, 3, 4, 5),
  Question1 = c("Agree", "Strongly Agree", "Neutral", "Agree", "Strongly Agree"),
  Question2 = c("Neutral", "Agree", "Agree", "Strongly Agree", "Agree"),
  Question3 = c("Strongly Agree", "Agree", "Neutral", "Agree", "Neutral")
)
names(survey_data) <- c(
  "Survey ID",
  "Question 1 Response",
  "Question 2 Response",
  "Question 3 Response"
)
print("Survey Response Data")
print(survey_data)

```

SurveyID	Question1	Question2	Question3
1	1	A	B
2	2	B	A
3	3	C	A

10.D: Develop a Tableau dashboard combining the grouped bar chart, stacked bar chart, and the table for interactive exploration of survey responses.

```
library(shiny)
```

```
survey_data <- data.frame(
  Survey_ID = c(1, 2, 3, 4, 5),
  Question1 = c("Agree", "Strongly Agree", "Neutral", "Agree", "Strongly Agree"),
  Question2 = c("Neutral", "Agree", "Agree", "Strongly Agree", "Agree"),
  Question3 = c("Strongly Agree", "Agree", "Neutral", "Agree", "Neutral")
)
```

```
ui <- fluidPage(
  titlePanel("Survey Response Dashboard"),

  sidebarLayout(
    sidebarPanel(
      selectInput(
        "response",
        "Select Response:",
        choices = c("All", "Neutral", "Agree", "Strongly Agree")
      )
    ),

```

```
    mainPanel(
      plotOutput("groupedBar"),
      plotOutput("stackedBar"),

```



```

    tableOutput("surveyTable")
  )
)
)
server <- function(input, output) {
  output$groupedBar <- renderPlot({
    response_levels <- c("Neutral", "Agree", "Strongly Agree")
    response_counts <- sapply(
      survey_data[, 2:4],
      function(x) table(factor(x, levels = response_levels))
    )
    barplot(
      response_counts,
      beside = TRUE,
      legend.text = response_levels,
      xlab = "Questions",
      ylab = "Number of Responses",
      main = "Grouped Bar Chart of Survey Responses"
    )
  })
  output$stackedBar <- renderPlot({
    response_levels <- c("Neutral", "Agree", "Strongly Agree")
    response_counts <- sapply(
      survey_data[, 2:4],
      function(x) table(factor(x, levels = response_levels))
    )
    barplot(
      response_counts,
      beside = FALSE,
      legend.text = response_levels,
      xlab = "Questions",

```

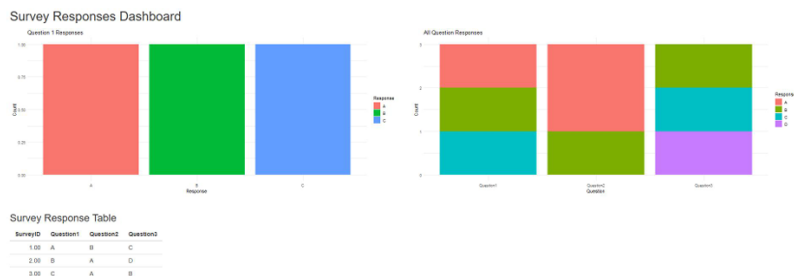
```

  ylab = "Number of Responses",
  main = "Overall Distribution of Survey Responses"
)
})

output$surveyTable <- renderTable({
  if (input$response == "All") {
    survey_data
  } else {
    survey_data[
      survey_data$Question1 == input$response |
      survey_data$Question2 == input$response |
      survey_data$Question3 == input$response,
    ]
  }
})
}

shinyApp(ui = ui, server = server)

```



SET 11:

11.A:Generate a funnel chart to analyze the sales conversion process for each product category. Label the stages and title the chart.

```
stages <- c("Visitors", "Product Views", "Add to Cart", "Checkout", "Purchases")
```

```
Electronics <- c(1000, 800, 500, 300, 200)
```

```
Clothing <- c(900, 700, 450, 250, 150)
```

```
Furniture <- c(700, 550, 350, 200, 100)
```

```
par(mfrow = c(1, 3))
```

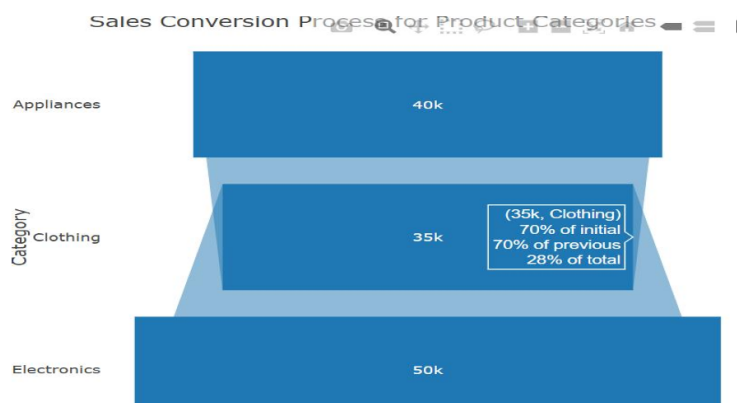
```
barplot(
```

Electronics

```

names.arg = stages,
horiz = TRUE,
main = "Electronics Sales Funnel",
xlab = "Number of Customers",
las = 1
)
barplot(
  Clothing,
  names.arg = stages,
  horiz = TRUE,
  main = "Clothing Sales Funnel",
  xlab = "Number of Customers",
  las = 1
)
barplot(
  Furniture,
  names.arg = stages,
  horiz = TRUE,
  main = "Furniture Sales Funnel",
  xlab = "Number of Customers",
  las = 1
)
par(mfrow = c(1, 1))

```



11.B:Build a table to display the sales data for each product category. Label the table elements.

```
sales_data <- data.frame(  
  Product_Category = c("Electronics", "Clothing", "Furniture"),  
  Visitors = c(1000, 900, 700),  
  Product_Views = c(800, 700, 550),  
  Add_to_Cart = c(500, 450, 350),  
  Checkout = c(300, 250, 200),  
  Purchases = c(200, 150, 100)  
)  
names(sales_data) <- c(  
  "Product Category",  
  "Visitors",  
  "Product Views",  
  "Add to Cart",  
  "Checkout",  
  "Purchases"  
)  
  
print("Sales Data by Product Category")  
print(sales_data)
```

Category Sales	
1	Electronics 50000
2	Clothing 35000
3	Appliances 40000

11.C:Develop a Tableau dashboard combining the pie chart, funnel chart, and the table for interactive exploration of product category data.

```
library(shiny)
```

```
sales_data <- data.frame(  
  Product_Category = c("Electronics", "Clothing", "Furniture")
```

```

Visitors = c(1000, 900, 700),
Product_Views = c(800, 700, 550),
Add_to_Cart = c(500, 450, 350),
Checkout = c(300, 250, 200),
Purchases = c(200, 150, 100)
)

ui <- fluidPage(
  titlePanel("Product Category Sales Dashboard"),

  sidebarLayout(
    sidebarPanel(
      selectInput(
        "category",
        "Select Product Category:",
        choices = c("All", sales_data$Product_Category)
      )
    ),

    mainPanel(
      plotOutput("pieChart"),
      plotOutput("funnelChart"),
      tableOutput("salesTable")
    )
  )
)

server <- function(input, output) {

  filtered_data <- reactive({
    if (input$category == "All") {

```

```
    sales_data
  } else {
    sales_data[sales_data$Product_Category == input$category, ]
  }
})
```

```
output$pieChart <- renderPlot({
  pie(
    filtered_data()$Purchases,
    labels = filtered_data()$Product_Category,
    main = "Purchases by Product Category"
  )
})
```

```
output$funnelChart <- renderPlot({
  data <- filtered_data()
```

```
  stages <- c(
    "Visitors",
    "Product Views",
    "Add to Cart",
    "Checkout",
    "Purchases"
  )
```

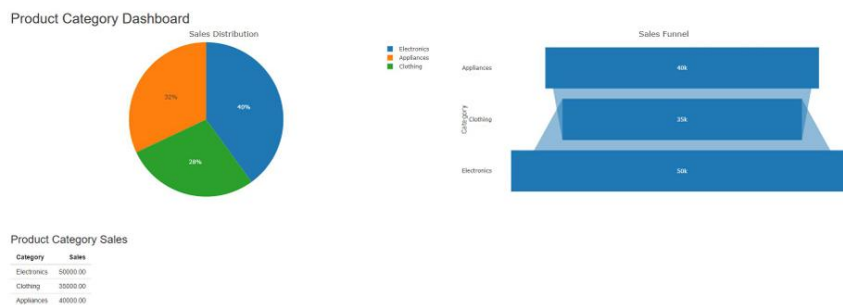
```
  values <- c(
    sum(data$Visitors),
    sum(data$Product_Views),
    sum(data$Add_to_Cart),
    sum(data$Checkout),
    sum(data$Purchases)
```

```
)
```

```
barplot(  
  values,  
  names.arg = stages,  
  horiz = TRUE,  
  las = 1,  
  xlab = "Number of Customers",  
  main = "Sales Conversion Funnel"  
)  
})
```

```
output$salesTable <- renderTable({  
  filtered_data()  
})  
}
```

```
shinyApp(ui = ui, server = server)
```



11.D: Create a pie chart to represent the distribution of sales across product categories. Include labels.

```
sales_data <- data.frame(  
  Product_Category = c("Electronics", "Clothing", "Furniture"),  
  Sales = c(200, 150, 100)  
)  
percent <- round(
```

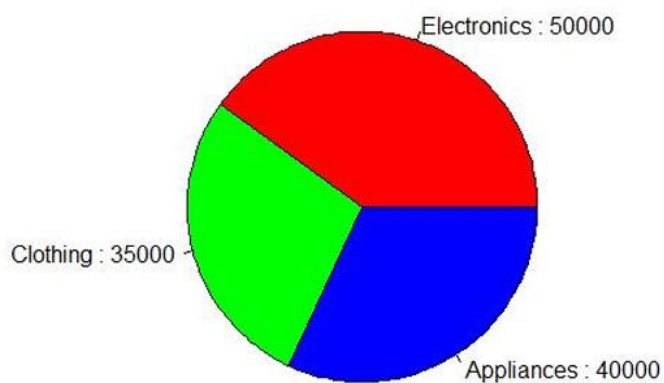
```

sales_data$Sales / sum(sales_data$Sales) * 100,
1
)
labels <- paste(
  sales_data$Product_Category,
  percent,
  "% ",
  sep = " "
)

pie(
  sales_data$Sales,
  labels = labels,
  main = "Distribution of Sales Across Product Categories"
)

```

Sales Distribution Across Product Categories



SET 12:

12.A: Create a line chart to visualize the trend in daily page views over time. Label the axes and title the chart.

```

website_data <- data.frame(
  Date = as.Date(c(
    "2023-01-01",
    "2023-01-02",

```



```

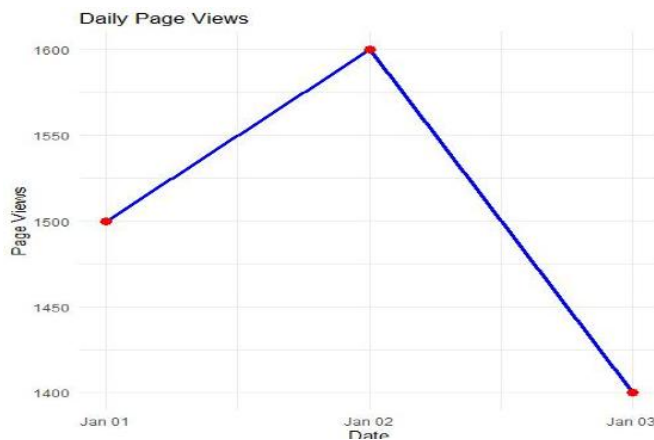
"2023-01-03",
"2023-01-04",
"2023-01-05"
)),
Page_Views = c(1500, 1600, 1400, 1650, 1800)
)

```

```

plot(
  website_data$Date,
  website_data$Page_Views,
  type = "o",
  pch = 16,
  xlab = "Date",
  ylab = "Page Views",
  main = "Trend in Daily Page Views Over Time"
)

```



12.B:Generate a bar chart showing the top N days with the highest click-through rates. Label the chart elements.

```

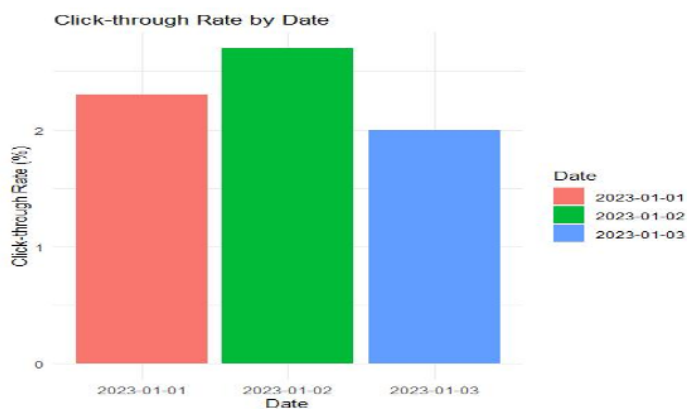
website_data <- data.frame(
  Date = as.Date(c(
    "2023-01-01",
    "2023-01-02",
    "2023-01-03",

```

```

"2023-01-04",
"2023-01-05"
)),
Click_Through_Rate = c(2.3, 2.7, 2.0, 2.4, 2.6)
)
N <- 3
top_days <- website_data[
  order(website_data$Click_Through_Rate, decreasing = TRUE),
][1:N, ]
bars <- barplot(
  top_days$Click_Through_Rate,
  names.arg = top_days$Date,
  xlab = "Date",
  ylab = "Click-Through Rate (%)",
  main = paste("Top", N, "Days with Highest Click-Through Rates")
)
text(
  bars,
  top_days$Click_Through_Rate,
  labels = paste0(top_days$Click_Through_Rate, "%"),
  pos = 3
)

```



12.C: Build a table to show the website traffic data for each day. Label the table elements.

```

website_data <- data.frame(
  Date = as.Date(c(
    "2023-01-01",
    "2023-01-02",
    "2023-01-03",
    "2023-01-04",
    "2023-01-05"
  )),
  Page_Views = c(1500, 1600, 1400, 1650, 1800),
  Click_Through_Rate = c(2.3, 2.7, 2.0, 2.4, 2.6)
)
names(website_data) <- c(
  "Date",
  "Page Views",
  "Click-Through Rate (%)"
)
print("Website Traffic Data")

```

```
print(website_data)
```

	Date	PageViews	CTR
1	2023-01-01	1500	2.3%
2	2023-01-02	1600	2.7%
3	2023-01-03	1400	2.0%

12.D:Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of website traffic data.

```
library(shiny)
```

```

website_data <- data.frame(
  Date = as.Date(c(
    "2023-01-01",
    "2023-01-02",
    "2023-01-03",

```

```
"2023-01-04",  
"2023-01-05"  
)),  
Page_Views = c(1500, 1600, 1400, 1650, 1800),  
Click_Through_Rate = c(2.3, 2.7, 2.0, 2.4, 2.6)  
)
```

```
ui <- fluidPage(  
  titlePanel("Website Traffic Dashboard"),
```

```
  sidebarLayout(  
    sidebarPanel(  
      sliderInput(  
        "N",  
        "Select Top N Days:",  
        min = 1,  
        max = 5,  
        value = 3  
      )  
    ),  
    mainPanel(  
      plotOutput("lineChart"),  
      plotOutput("barChart"),  
      tableOutput("trafficTable")  
    )  
  )  
)
```

```
server <- function(input, output) {  
  output$lineChart <- renderPlot({  
    plot(  
      website_data$Date,
```

```

website_data$Page_Views,
type = "o",
pch = 16,
xlab = "Date",
ylab = "Page Views",
main = "Daily Page Views Over Time"
)
})
output$barChart <- renderPlot({
top_days <- website_data[
order(website_data$Click_Through_Rate, decreasing = TRUE),
][1:input$N, ]
barplot(
top_days$Click_Through_Rate,
names.arg = top_days$Date,
xlab = "Date",
ylab = "Click-Through Rate (%)",
main = "Top Days with Highest Click-Through Rates"
)
})
output$trafficTable <- renderTable({
website_data
})
}
shinyApp(ui = ui, server = server)

```

